MODULE 1

INTRODUCTION

CONTENT

- Concept & Overview of DBMS
- Data Models
- Database Languages
- Database Administrator, Database Users
- Three Schema architecture of DBMS

CONCEPT OF DBMS

- **Database**: A collection of related data.
- **Data**: Known facts that can be recorded and have an implicit meaning.
- **Mini-world**: Some part of the real world about which data is stored in a database. For example, student grades and transcripts at a university.
- **Database Management System (DBMS)**: A software package/ system to facilitate the creation and maintenance of a computerized database.
- **Database System**: The DBMS software together with the data itself. Sometimes, the applications are also included.

DEFINITION OF DBMS

In a nutshell,DBMS can be defined as

- Collection of interrelated data
- Set of programs to access the data
- DBMS contains information about a particular enterprise
- DBMS provides an environment that is both *convenient* and *efficient* to use.

Typical DBMS Functionality

- Define a database : in terms of data types, structures and constraints
- Construct or Load the Database on a secondary storage medium
- Manipulating the database : querying, generating reports, insertions, deletions and modifications to its content
- Concurrent Processing and Sharing by a set of users and programs – yet, keeping all data valid and consistent

Application of Database Management System (DBMS)

- Database Applications:
  - Banking: all transactions
  - Airlines: reservations, schedules
  - Universities: registration, grades
  - Sales: customers, products, purchases
  - Manufacturing: production, inventory, orders, supply chain
  - Human resources: employee records, salaries, tax deductions
- Databases touch all aspects of our lives

Purpose of Database System

- In the early days, database applications were built on top of file systems
- Drawbacks of using file systems to store data:
  - Data redundancy and inconsistency
    - Multiple file formats, duplication of information in different files
  - Difficulty in accessing data
    - Need to write a new program to carry out each new task
  - Data isolation — multiple files and formats
  - Integrity problems
    - Integrity constraints (e.g. account balance > 0) become part of program code
    - Hard to add new constraints or change existing ones

Purpose of Database Systems (Cont.)

- Drawbacks of using file systems (cont.)
  - Atomicity of updates
    - Failures may leave database in an inconsistent state with partial updates carried out
    - E.g. transfer of funds from one account to another should either complete or not happen at all
  - Concurrent access by multiple users
    - Concurrent accessed needed for performance
    - Uncontrolled concurrent accesses can lead to inconsistencies
      - E.g. two people reading a balance and updating it at the same time
  - Security problems
- Database systems offer solutions to all the above problems

Database Schema, Instance and State

- Database Schema:
  - Includes descriptions of the database structure, data types, and the constraints on the database.
- Schema Diagram:
  - An ***illustrative*** display of (most aspects of) a database schema.
- Schema Construct:
  - A ***component*** of the schema or an object within the schema, e.g., STUDENT, COURSE.
- Database State:
- Distinction
  - The ***database schema*** changes very infrequently.
  - The ***database state*** changes every time the database is updated.
- **Schema** is also called **intension**.
- **State** is also called **extension**.

Levels of Abstraction

- Physical level describes how a record (e.g., customer) is stored.
- Logical level: describes data stored in database, and the relationships among the data.

```
type customer = record
                    name : string;
                    street : string;
                    city : integer;
                end;
```

- View level: application programs hide details of data types. Views can also hide information (e.g., salary) for security purposes.

Database Users

Users are differentiated by the way they expect to interact with the system

- **Application programmers** – interact with system through DML calls
- **Sophisticated users** – form requests in a database query language
- **Specialized users** – write specialized database applications that do not fit into the traditional data processing framework
- **Naive users** – invoke one of the permanent application programs that have been written previously
  - E.g. people accessing database over the web, bank tellers, clerical staff

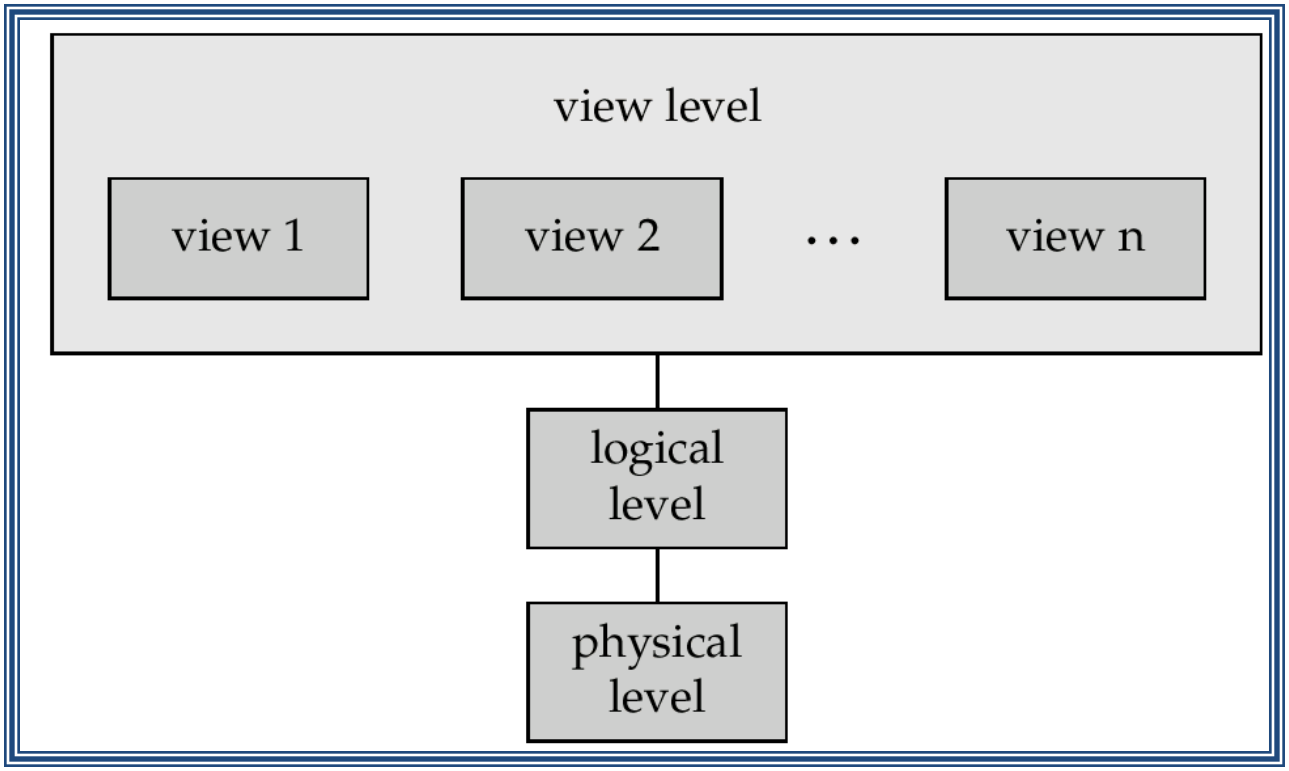
Database Administrator

- Coordinates all the activities of the database system; the database administrator has a good understanding of the enterprise's information resources and needs.
- Database administrator's duties include:
  - Schema definition
  - Storage structure and access method definition
  - Schema and physical organization modification
  - Granting user authority to access the database
  - Specifying integrity constraints
  - Acting as liaison with users
  - Monitoring performance and responding to changes in requirements

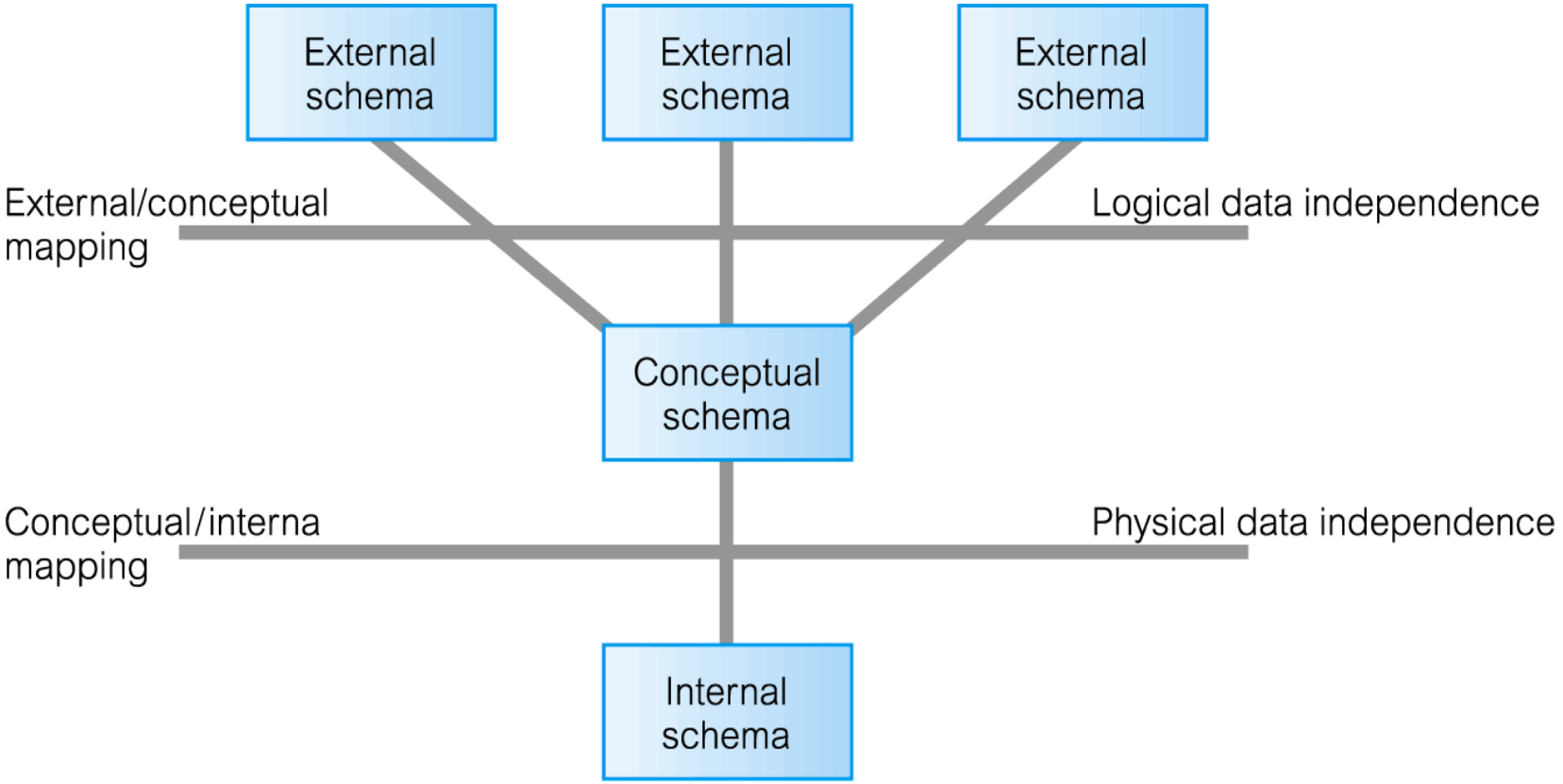
Objectives of Three-Level Architecture

- DBA should be able to change database storage structures without affecting the users' views.
- Internal structure of database should be unaffected by changes to physical aspects of storage.
- DBA should be able to change conceptual structure of database without affecting all users.

THREE LEVEL ARCHITECTURE



DATA INDEPENDENCE



Data Independence

- **Logical Data Independence**
  - Refers to immunity of external schemas to changes in conceptual schema.
  - Conceptual schema changes (e.g. addition/removal of entities).
  - Should not require changes to external schema or rewrites of application programs**.**

Data Independence

- **Physical Data Independence**
  - Refers to immunity of conceptual schema to changes in the internal schema.
  - Internal schema changes (e.g. using different file organizations, storage structures/devices).
  - Should not require change to conceptual or external schemas.

DATABASE LANGUAGES

- **Data Definition Language (DDL)**
  - Allows the DBA or user to describe and name entities, attributes, and relationships required for the application
  - plus any associated integrity and security constraints.

Database Languages

- Data Manipulation Language (DML)
  - Provides basic data manipulation operations on data held in the database.
- Procedural DML
  - allows user to tell system exactly how to manipulate data.
- Non-Procedural DML
  - allows user to state what data is needed rather than how it is to be retrieved.
- Fourth Generation Languages (4GLs)

DBMS Programming Language Interfaces

- **Embedded Approach**: e.g embedded SQL (for C, C++, etc.), SQLJ (for Java)
- **Procedure Call Approach**: e.g. JDBC for Java, ODBC for other programming languages
- **Database Programming Language Approach**: e.g. ORACLE has PL/SQL
- Menu-based, popular for browsing on the web
- Forms-based, designed for naïve users
- Graphics-based
- Natural language: requests in written English
- Combinations of the above:
- Speech as Input and Output
- Web Browser as an interface
- Parametric interfaces, e.g., bank tellers using function keys.
- Interfaces for the DBA:

Database System Utilities

- To perform certain functions such as:
  - Loading data stored in files into a database. Includes data conversion tools.
  - Backing up the database periodically on tape.
  - Reorganizing database file structures.
  - Report generation utilities.
  - Performance monitoring utilities.
- Data dictionary / repository:
- Application Development Environments and CASE (computer-aided software engineering) tools:
- Examples:
  - PowerBuilder (Sybase)
  - JBuilder (Borland)
  - JDeveloper 10G (Oracle)

DATA MODELS

• Representation of reality, ‘real world’ objects and events, and their associations.

•An abstraction that concentrates on the essential, inherent aspects of an organization.

•Can be defined as an integrated collection of concepts for describing and manipulating data, relationships between data, and constraints on the data in an organization.

COMPONENTS OF DATA MODEL

A data model comprises of three components:

- A structural part, consisting of a set of rules according to which databases can be constructed.
- A manipulative part, defining the types of operation that are allowed on the data (this includes the operations that are used for updating or retrieving data from the database and for changing the structure of the database).
- Possibly a set of integrity rules, which ensures that the data is accurate.

CATEGORIES OF DATA MODEL

The purpose of a data model is to represent data and to make the data understandable. There have been many data models proposed in the literature. They fall into three broad categories:

- Object Based Data Models
- Physical Data Models
- Record Based Data Models

The object based and record based data models are used to describe data at the conceptual and external levels, the physical data model is used to describe data at the internal level.

OBJECT BASED DATA MODELS

Object based data models use concepts such as entities, attributes, and relationships. An entity is a distinct object (a person, place, concept, event) in the organization that is to be represented in the database. An attribute is a property that describes some aspect of the object that we wish to record, and a relationship is an association between entities.

Some of the more common types of object based data model are:

- Entity—Relationship
- Object Oriented

The Entity-Relationship model has emerged as one of the main techniques for modeling database design and forms the basis for the database design methodology. The object oriented data model extends the definition of an entity to include, not only the attributes that describe the state of the object but also the actions that are associated with the object, that is, its behavior.

Physical Data Models

Physical data models describe how data is stored in the computer, representing information such as record structures, record ordering, and access paths. There are not as many physical data models as logical data models, the most common one being the Unifying Model.

Record Based Logical Models

Record based logical models are used in describing data at the logical and view levels. In contrast to object based data models, they are used to specify the overall logical structure of the database and to provide a higher-level description of the implementation. The three most widely accepted record based data models are:

- Hierarchical Model
- Network Model
- Relational Model

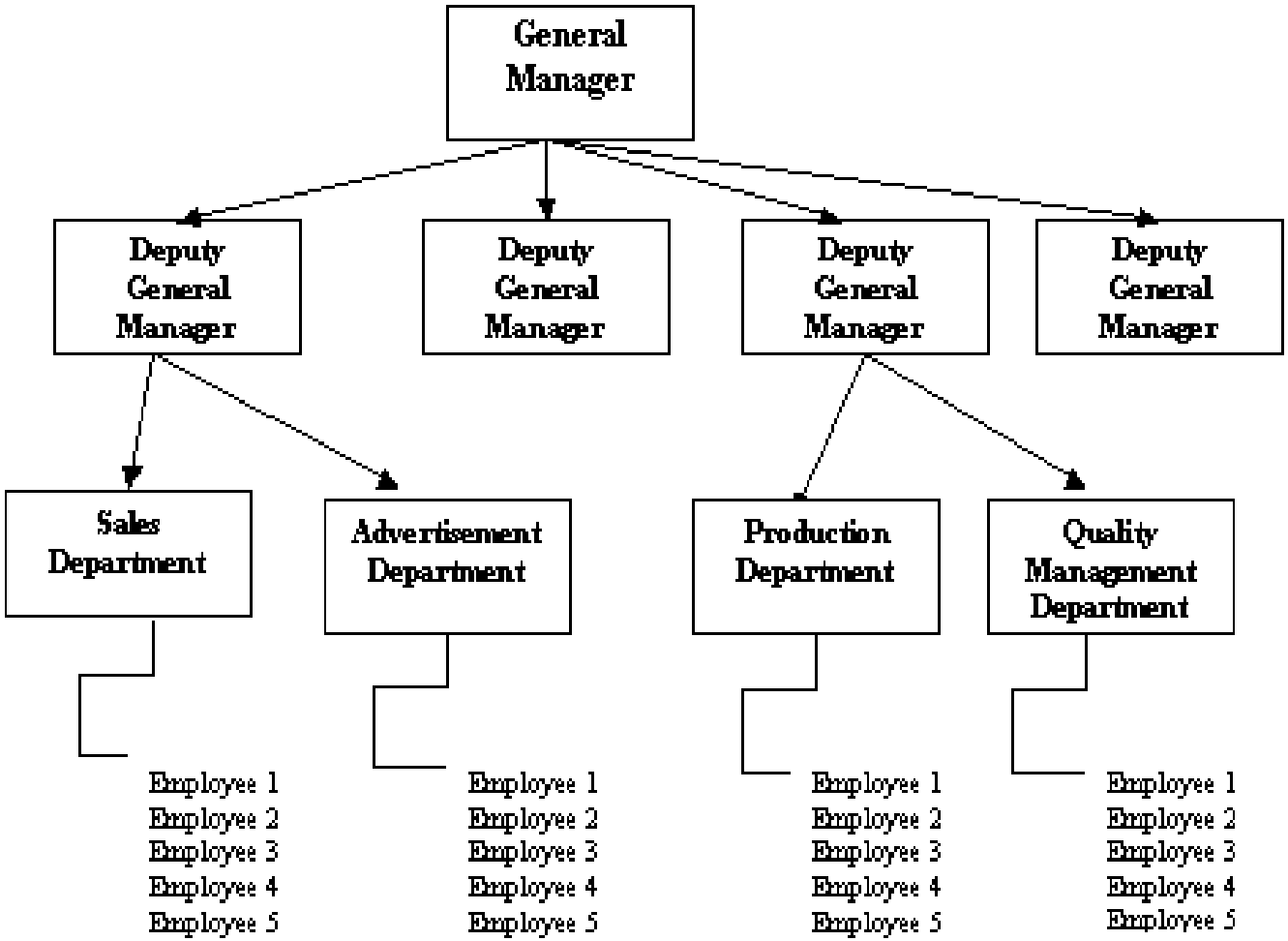
HIERARCHICAL MODEL

Hierarchical Database model is one of the oldest database models. This model is like a structure of a tree with the records forming the nodes and fields forming the branches of the tree.

Operations on Hierarchical Model

- **Insert Operation**
- **Update Operation**
- **Delete Operation**

EXAMPLE OF HIERARCHICAL MODEL



Advantages of Hierarchical Model

- **Simplicity:**
- **Data Security:**
- **Data Integrity:**
- **Efficiency**

The hierarchical database model is a very efficient one when the database contains a large number of 1:N relationships (one-to-many relationships) and when the users require large number of transactions, using data whose relationships are fixed.

DISADVANTAGE OF HIERARCHICAL MODEL

- **Implementation Complexity:**
- **Database Management Problems:** If you make any changes in the database structure of a hierarchical database, then you need to make the necessary changes in all the application programs that access the database.
- **Lack of structural independence:** Structural independence exists when the changes to the database structure does not affect the DBMS's ability to access data. Hierarchical database systems use physical storage paths to navigate to the different data segments. So the application programs should have a good knowledge of the relevant access paths to access the data.

Disadvantage of Hierarchical Model(contd.)

- **Operational Anomalies:** As discussed earlier, hierarchical model suffers from the Insert anomalies, Update anomalies and Deletion anomalies, also the retrieval operation is complex and asymmetric, thus hierarchical model is not suitable for all the cases.

- **Implementation Limitation:** Many of the common relationships do not conform to the 1:N format required by the hierarchical model. The many-to-many (N:N) relationships, which are more common in real life are very difficult to implement in a hierarchical model.

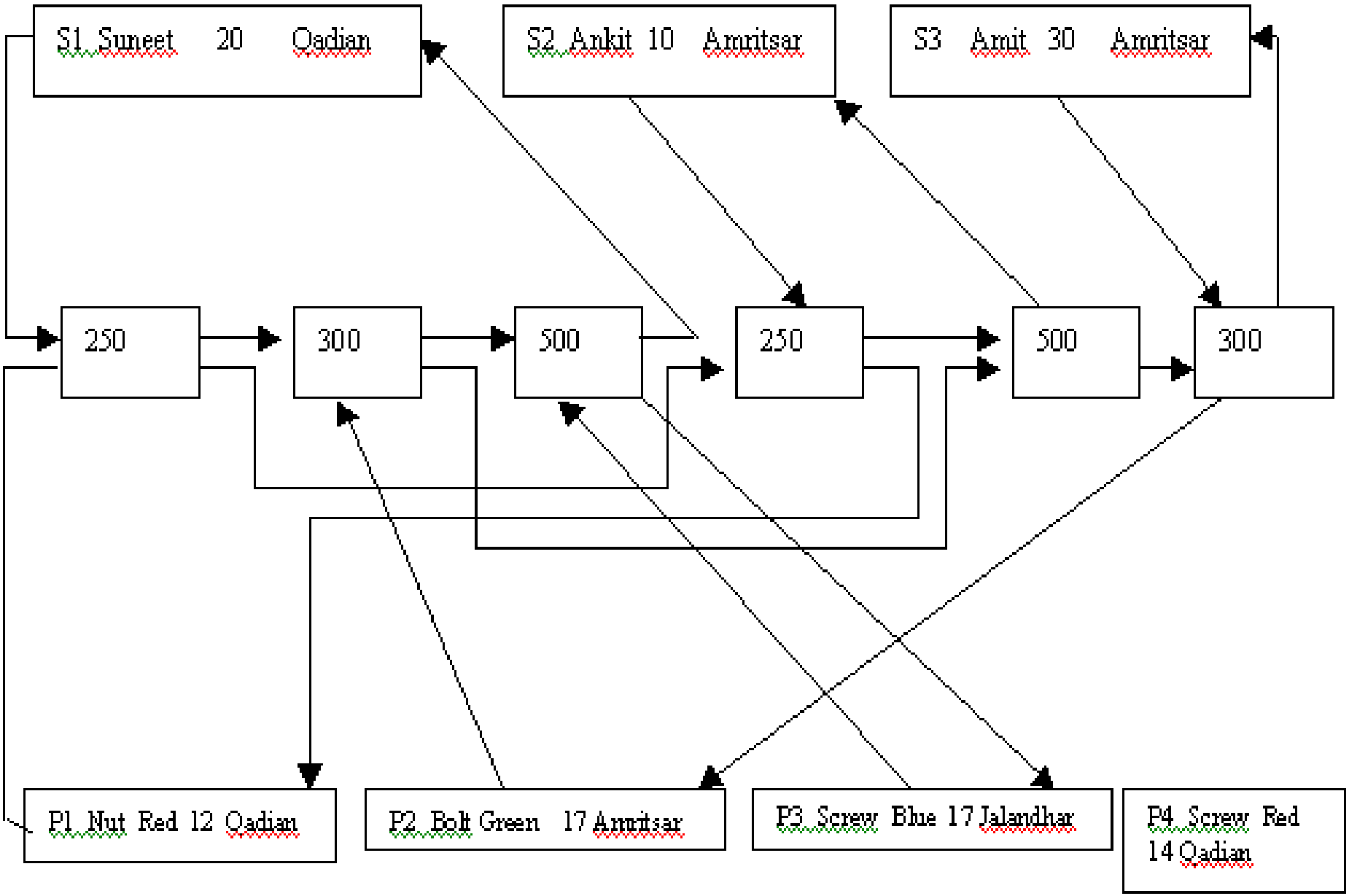
NETWORK MODEL

• The Network model replaces the hierarchical tree with a graph thus allowing more general connections among the nodes. The main difference of the network model from the hierarchical model, is its ability to handle many to many (N:N) relations.

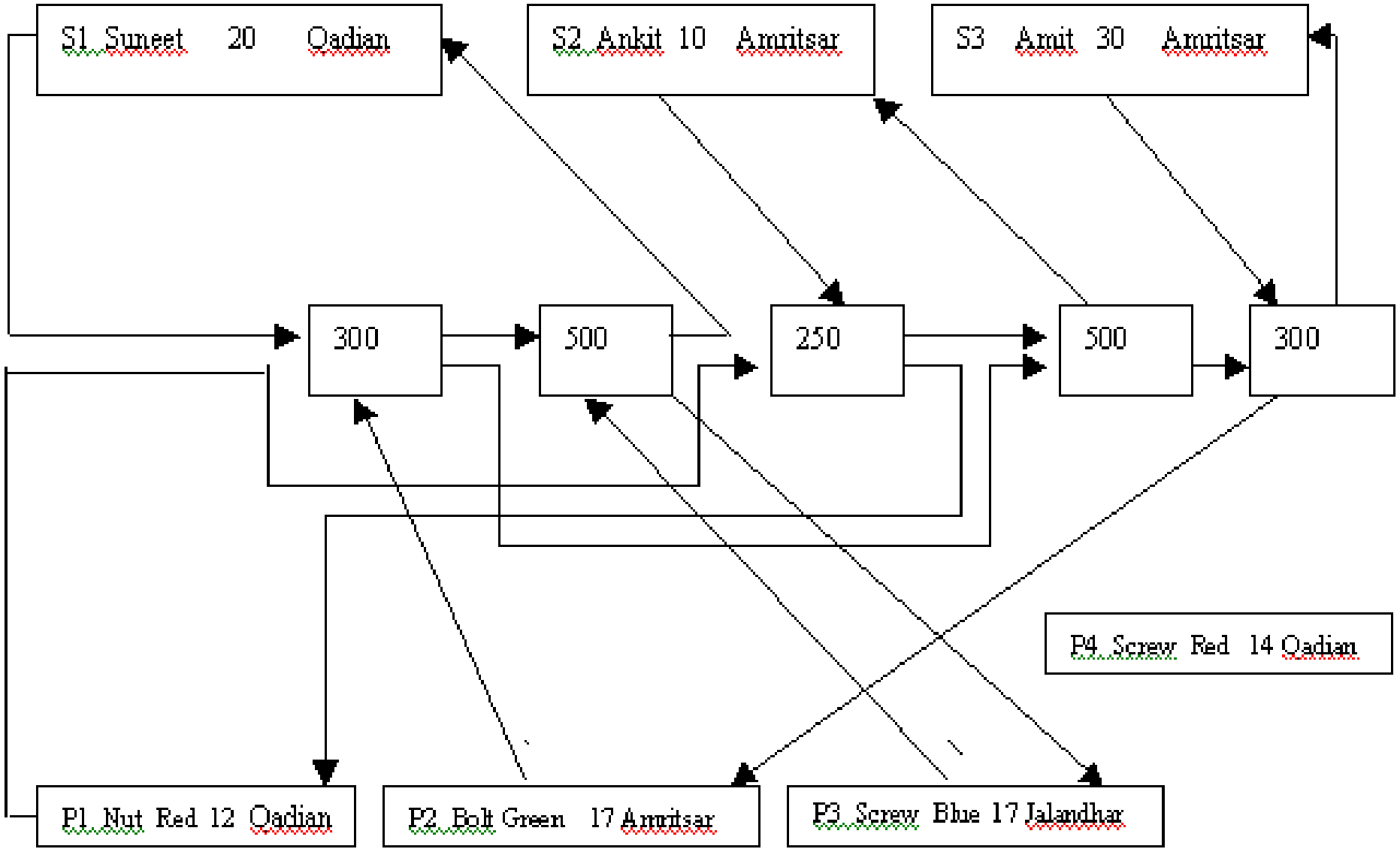
• In other words, it allows a record to have more than one parent. Suppose an employee works for two departments. The strict hierarchical arrangement is not possible here and the tree becomes a more generalized graph – a network. The network model was evolved to specifically handle non-hierarchical relationships. A network structure thus allows 1:1 (one:one), 1:M (one:many), M:M (many:many) relationships among entities.

• In network database terminology, a relationship is a set. Each set is made up of at least two types of records: an owner record (equivalent to parent in the hierarchical model) and a member record (similar to the child record in the hierarchical model).

NETWORK VIEW OF SAMPLE



For example if supplier S1 stops the supply of part P1 with 250 quantity the model is modified without affecting P1 and S1 information.



Advantages of Network Model

- **Conceptual simplicity:**

- **Capability to handle more relationship types:**

The network model can handle the one-to-many (1:N) and many to many (N:N) relationships, which is a real help in modeling the real life situations.

- **Ease of data access:**

The data access is easier than and flexible than the hierarchical model.

- **Data Integrity:**

The network model does not allow a member to exist without an owner. Thus a user must first define the owner record and then the member record. This ensures the data integrity.

DATA INDEPENDENCE

The network model is better than the hierarchical model in isolating the programs from the complex physical storage details.

DATABASE STANDARDS

One of the major drawbacks of the hierarchical model was the non-availability of universal standards for database design and modeling. The network model is based on the standards formulated by the DBTG and augmented by ANSI/SPARC (American National Standards Institute/Standards Planning and Requirements Committee) in the 1970s. All the network database management systems conformed to these standards. These standards included a Data Definition Language (DDL) and the Data Manipulation Language (DML), thus greatly enhancing database administration and portability.

DISADVANTAGES OF NETWORK MODEL

- **System complexity:**

All the records are maintained using pointers and hence the whole database structure becomes very complex.

- **Operational Anomalies:**

As discussed earlier, network model's insertion, deletion and updating operations of any record require large number of pointer adjustments, which makes its implementation very complex and complicated.

- **Absence of structural independence:**

Since the data access method in the network database model is a navigational system, making structural changes to the database is very difficult in most cases and impossible in some cases. If changes are made to the database structure then all the application programs need to be modified before they can access data.

RELATIONAL MODEL

Relational model stores data in the form of tables. The relational model consists of three major components:

1. The set of relations and set of domains that defines the way data can be represented (data structure).

2. Integrity rules that define the procedure to protect the data (data integrity).

3. The operations that can be performed on data (data manipulation).

A relational model database is defined as a database that allows you to group its data items into one or more independent tables that can be related to one another by using fields common to each related table.

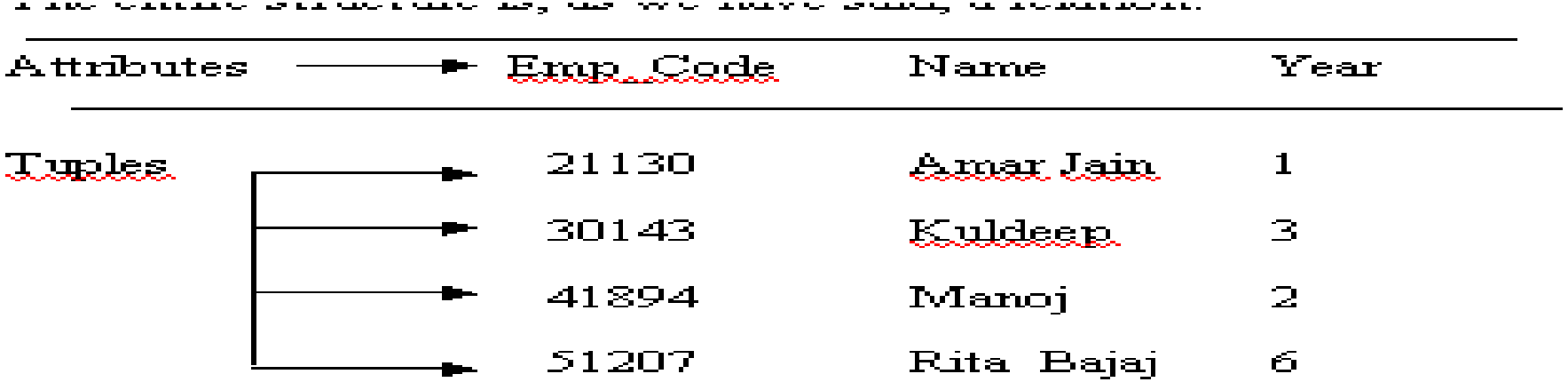
Characteristics of Relational Database

The whole data is conceptually represented as an orderly arrangement of data into rows and columns, called a relation or table.

All values are scalar. That is, at any given row/column position in the relation there is one and only one value.

All operations are performed on an entire relation and result is an entire relation, a concept known as closure.

BASIC TERMINOLOGY USED IN RELATIONAL MODEL



Tuples of a Relation

Cardinality of a relation

The number of tuples in a relation determines its cardinality. In this case, the relation has a cardinality of 4.

Degree of a relation Each column in the tuple is called an attribute. The number of attributes in a relation determines its degree. The relation in has a degree of 3.

ADVANTAGES OF RELATIONAL MODEL

Structural independence:

In relational model, changes in the database structure do not affect the data access. When it is possible to make change to the database structure without affecting the DBMS's capability to access data, we can say that structural independence have been achieved. So, relational database model has structural independence.

Conceptual simplicity

Design, implementation, maintenance and usage ease

Ad hoc query capability:

The presence of very powerful, flexible and easy-to-use query capability is one of the main reasons for the immense popularity of the relational database model. The query language of the relational database models structured query language or SQL makes ad hoc queries a reality. **SQL is a fourth generation language (4GL).**

DISADVANTAGES OF RELATIONAL MODEL

The drawbacks of the relational database systems could be avoided if proper corrective measures are taken. The drawbacks are not because of the shortcomings in the database model, but the way it is being implemented.

- **Hardware overheads**
- **Ease of design can lead to bad design**
- **‘Information island’ phenomenon**

COMPARISONS AMONG THREE DATA MODELS

S.No	Hierarchical Data Model	Network Data Model	Relational Data Model
1.	Relationship between records is of the parent child type	Relationship between records is expressed in the form of pointers or links	Relationship between records is represented by a relation that contains a key for each record involved in the relationship
2.	Many to many relationship cannot be expressed in this model	Many to many relationship can also be implemented	Many to many relationship can be easily implemented
3.	It is a simple, straightforward and natural method of implementing record relationships	Record relationship implementation is very complex due to the use of pointers	Relationship implementation is very easy through the use of a key or composite key field(s)
4.	This type of model is useful only when there is some hierarchical character in the database.	Network model is useful for representing such records which have many to many relationships	Relational model is useful for representing most of the real world objects and relationships among them
5.	In order to represent links among records, pointers are used. Thus relations among records are physical.	In Network model also the record relations are physical.	Relational model does not maintain physical connection among records

END
OF
MODULE 1